

The Six Plugins, Explained

A technical rundown of every Prometheus plugin — what it is, how it works, and why it beats the way factories do it today.

Prometheus is the manufacturing plugin suite that runs on the Atlas Framework. Every plugin is called by the same local orchestrator (Vulcan, an 8B-parameter kernel DPO-tuned from Gemma 4), and every plugin is 100% offline — no data ever leaves the factory.

The six plugins address all four Luxshare challenge categories: **AD** (anomaly detection), **TE** (test engineering), **MLB** (mainboard / process engineering), and **IT** (data / compliance).

#	PLUGIN	ROLE	COVERS
01	Manufacturing World Model	Predicts drift before defect	AD
02	Causal Discovery Engine	Finds the cause, not just the correlation	AD + MLB
03	Physics-Accelerated Simulator	12-hour CFD → 50 ms PINN surrogate	MLB + TE
04	Autonomous Sandbox Lab	Self-driving virtual experiments	AD + MLB
05	Offline Vision Inspector	On-prem VLM replaces L2 human reviewer	TE + MLB
06	Traceability & Compliance Engine	Typed knowledge graph, signed audits	IT

Manufacturing World Model

Teach an AI what a healthy factory sounds like — then have it listen for anything off.

THE IDEA

A factory has thousands of sensors — temperature probes in the oven, vibration sensors on the conveyor, force sensors on the robot arms. Every second, each one spits out a number. That's **time-series data**: numbers that have an order. MWM learns the normal patterns in those numbers and flags anything that breaks the pattern.

WHAT'S ACTUALLY HAPPENING

We feed millions of hours of sensor data into a **Time-Series Foundation Model** (Chronos + IBM-TTM lineage). Think of it like GPT, except instead of predicting the next word, it predicts the next *sensor reading*. After training, it has an internal sense of 'normal.'

When new data comes in, we ask the model what it *expected* the next 60 minutes to look like. Then we compare that prediction to what actually happened. The distance between the two is measured with the **Mahalanobis distance** — a smarter version of Z-score that accounts for how sensors correlate with each other. If it crosses 2σ , we flag it:

$$\text{score}(t) = \| x(t : t+h) - \hat{x}(t : t+h) \| \text{ over } \Sigma^{-1} \hat{x} \sim p_{\theta} (\cdot | \text{past})$$

x = real sensor readings · $\hat{x}$ = model prediction · Σ^{-1} = inverse covariance (accounts for sensor correlation)

WHY IT BEATS TRADITIONAL

Classical Statistical Process Control (SPC) uses univariate control charts — one sensor, one threshold — so it misses *joint* drift patterns and only fires *after* a defect appears. Defects almost never appear out of nowhere. Before a bad solder joint, the oven temp drifts a hair, the conveyor speeds up, humidity shifts. Each one alone is invisible, but together they're a pattern. MWM sees the pattern and fires up to **60 minutes before the first bad unit** rolls off the line.

Causal Discovery Engine

Teach the AI the difference between 'these happen together' and 'this caused that.'

THE IDEA

Correlation is easy — any middle schooler with Excel can find it. The problem is that correlation lies. Ice cream sales correlate with drownings, but ice cream doesn't cause drownings — hot weather causes both. In a factory, mistaking a confounder for a cause burns weeks.

WHAT'S ACTUALLY HAPPENING

We use two tools together:

1. Physics-shaped priors. We assume defects follow real chemistry — the **Arrhenius equation** (reactions go exponentially faster as temperature rises) combined with a **power law** (some parameter matters to the β th power). Fitting this structured equation instead of a generic curve forces the model to find *physically plausible* drivers.

2. Interventional reasoning. We ask counterfactual questions: 'If I *force* humidity to stay constant, does the defect rate still track temperature?' If yes → temperature is a real cause. If no → humidity was the confounder all along.

$$\text{defect rate} \approx \alpha \cdot (T - T_0)^\beta \cdot \exp(-E_a / k_B \cdot T) \leftarrow \text{Arrhenius} \times \text{power-law}$$

T_0 = reference temperature · E_a = activation energy · k_B = Boltzmann constant

WHY IT BEATS TRADITIONAL

Traditional yield analysis stops at correlation heatmaps and Pareto charts: '20 variables correlate with failures, good luck.' Causal discovery tells you **which one, if you changed it, would actually fix the problem**. That's the difference between a hypothesis and a fix.

Physics-Accelerated Simulator

Replace a 12-hour physics simulation with a neural network that gives the same answer in 50 milliseconds.

THE IDEA

When you want to know how heat flows through a reflow oven, engineers solve **partial differential equations (PDEs)** — Navier-Stokes for airflow, the heat equation for temperature. These are the same equations used to model weather and jet engines. Doing this on a fine grid (Computational Fluid Dynamics, CFD) takes 8–12 hours per configuration.

WHAT'S ACTUALLY HAPPENING

We train a **Physics-Informed Neural Network (PINN)** on NVIDIA Modulus. A PINN is a regular neural net with a twist in the loss function: we take the net's output and plug it *back into the PDE*. If the output satisfies the physics, that loss term is zero. So the network can't just memorize — it has to learn physics-consistent behavior:

$$L(\theta) = \lambda_{\text{data}} \cdot L_{\text{data}} + \lambda_{\text{pde}} \cdot \|N[\hat{u}_{\theta}]\|^2 + \lambda_{\text{bc}} \cdot L_{\text{bc}}$$

L_{data} = fit training examples · $\|N[\hat{u}_{\theta}]\|^2$ = PDE residual (physics loss) · L_{bc} = boundary conditions

WHY IT BEATS TRADITIONAL

A plain neural net can overfit and give nonsense outside its training range. A PINN can't — the physics loss punishes any output that violates real physical law. So it's as fast as a neural net (**under 50 ms**) but as trustworthy as CFD. That's roughly a **10,000× speedup**, which means engineers can run live what-if sweeps on the factory floor instead of waiting overnight.

Autonomous Sandbox Lab

A robot scientist that runs experiments, learns from results, picks the next one — 300 times a day.

THE IDEA

You have knobs to tune — oven temp, airflow, conveyor speed — maybe 20 of them. Each combination is one 'experiment.' The parameter space is astronomical. Random search is wasteful. Grid search explodes combinatorially. You need a *smart* search.

WHAT'S ACTUALLY HAPPENING

We use **Bayesian optimization** with a library called **BoTorch**. The loop:

1. Model. Fit a **Gaussian Process (GP)** over the parameter space. At any point, a GP tells you two things: (a) its best-guess yield, and (b) how confident it is. Unexplored regions have huge error bars; regions near past experiments have tight ones.

2. Pick. Use the **Expected Improvement** acquisition function — 'how much better than the current best do I expect this point to be, weighted by uncertainty?' Pick the x that maximizes it:

$$\alpha_{EI}(x) = \sigma(x) \cdot [z \cdot \Phi(z) + \phi(z)] \quad z = (\mu(x) - f^*) / \sigma(x)$$

$\mu(x)$ = GP mean · $\sigma(x)$ = GP uncertainty · f^* = best result so far · Φ, ϕ = normal CDF and PDF

WHY IT BEATS TRADITIONAL

Bayesian optimization explores *smartly* — it balances exploiting promising regions against exploring uncertain ones. It's the same math that picks drug candidates and tunes language models. Each chosen point is evaluated against the MWM or the Physics Simulator, not a physical coupon. **~300 virtual trials per day** beats a human engineer running physical trials for months.

Offline Vision Inspector

A mini GPT-4V that runs on-prem and replaces the human who reviews flagged AOI images.

THE IDEA

Luxshare's factory already has an AOI (Automated Optical Inspection) camera that flags suspicious units. Problem: about 60% of those flags are false positives, so every flagged image goes to a human Level-2 reviewer. That's slow, expensive, and inconsistent between shifts.

WHAT'S ACTUALLY HAPPENING

We run **Qwen2-VL-2B**, an open-weights **vision-language model**. Vision-language models take an image *and* text as input and produce text as output — GPT-4V is the famous one; Qwen2-VL-2B is small enough (2 billion parameters) to run on a single edge GPU.

We prompt it with the flagged image and the part spec, and ask it to return a **structured JSON verdict**. The verdict goes back to the AOI controller over RS-232 / MQTT — the same serial protocols factory equipment has used since the 1990s:

```
verdict ∈ { confirmed , false_positive , needs_human } + rationale + bbox
```

Structured output that slots directly into existing AOI controller PLCs — no middleware rewrite.

WHY IT BEATS TRADITIONAL

Images of customer products (Apple, etc.) can't legally leave the building for IP reasons, so cloud GPT-4V is off the table. A local 2B-parameter model gets Luxshare the smart reviewer without the cloud call. Result: **~60% false-positive reduction, zero cloud calls**, IP stays on-prem.

Traceability & Compliance Engine

Turn every unit's life story into a knowledge graph — so an audit becomes a graph walk, not a week of SQL.

THE IDEA

Today, tracing 'which lot of copper went into serial #8842?' means joining MES (manufacturing execution), ERP (resource planning), and QMS (quality management). That's three databases, three schemas, and usually three different teams. A recall investigation takes weeks.

WHAT'S ACTUALLY HAPPENING

We build a **typed knowledge graph** using **Rustworkx**, a Rust-backed graph library for Python. Every real-world thing — a raw-material lot, an operator, a machine, a test result, a finished unit — becomes a **node**. Every relationship — USED_BY, OPERATED, TESTED_AT — becomes a **typed edge**.

A compliance query becomes a **typed sub-graph extraction**: 'give me every node within k typed edges of unit v_8842.' That's the full ancestry — lots, operators, tests — returned in seconds:

$$S(v_{8842}, k) = \{u \in V : d_{\text{typed}}(u, v_{8842}) \leq k\}$$

d_{typed} = typed graph distance · V = graph vertices · k = hop radius (how far back to trace)

WHY IT BEATS TRADITIONAL

Vulcan (the orchestrator) then reads the sub-graph and generates an ISO-compliant PDF. We hash the snapshot with **SHA-256** so a regulator can cryptographically verify the report wasn't tampered with after the fact. Weeks of manual SQL joins become **seconds of typed graph walk** with a signed, tamper-evident audit trail.

SUMMARY

The Six Plugins at a Glance

#	PLUGIN	CORE TECH	REPLACES	WIN
01	MWM	Time-series foundation model + Mahalanobis	Univariate SPC charts	Catches drift 60 min early
02	Causal Engine	Arrhenius × power law + do-calculus	Correlation Pareto charts	Finds causes , not co-occurrences
03	Physics Sim	PINN (NVIDIA Modulus)	Traditional CFD	~10,000× speedup
04	Sandbox Lab	Bayesian opt · BoTorch + GP + EI	Human-driven DOE	300 trials/day
05	Vision Inspector	Qwen2-VL-2B (on-prem)	Level-2 human reviewer	–60% FP , 0 cloud calls
06	Compliance Engine	Rustworkx typed KG + SHA-256	MES/ERP/QMS SQL joins	Weeks → seconds , signed

ONE-LINERS (IF YOU GET RUSHED)

MWM	Sees the defect 60 minutes before it happens.
Causal Engine	Tells you <i>why</i> , not just <i>what</i> .
Physics Sim	10,000× faster than CFD — still physics-valid.
Sandbox Lab	A self-driving lab, 300 experiments a day.
Vision Inspector	Replaces the L2 reviewer. Zero cloud. –60% false positives.
Compliance	Weeks of audit joins → seconds of graph walk, signed.